

KRIOS – FULL REBUILD PLAN

Logic First. UI Second. Performance Always.

TECH STACK CONFIRMED

Layer	Technology
Framework	Expo (React Native)
Navigation	Expo Router
Backend	Express + Node.js (local → Railway)
Database	Supabase
Real-time	Socket.io
AI	Vercel AI SDK
Animations	React Native Reanimated
Gestures	React Native Gesture Handler
Target devices	Low-budget Android + iOS

PACKAGES TO INSTALL / VERIFY

Run this to check what you already have:

```
cat package.json | grep dependencies -A 100
```

Required — install if missing:

```
npx expo install @gorhom/bottom-sheet
npx expo install react-native-pager-view
npx expo install react-native-keyboard-
controller
npx expo install @shopify/flash-list
npx expo install expo-image-picker
npx expo install expo-document-picker
npx expo install expo-av
npx expo install expo-location
npx expo install expo-haptics
npx expo install zustand
npx expo install @react-native-async-
storage/async-storage
npx expo install react-native-mmkv
```

Already confirmed installed:

- react-native-reanimated 
- react-native-gesture-handler 

Why these packages:

- **@gorhom/bottom-sheet** — for the swipe-down profile panel in chat (despite the name, it handles swipe-from-top via inverted config)
 - **react-native-pager-view** — for smooth simultaneous-mounted tab switching (fixes the swipe navigation mounting issue)
 - **@shopify/flash-list** — CRITICAL for low-budget phones. Much faster than FlatList for long message lists
 - **react-native-keyboard-controller** — for perfectly aligned input above keyboard on all devices
 - **react-native-mmkv** — fast key-value storage for bubble style preference and settings (much faster than AsyncStorage)
 - **zustand** — lightweight state management, no boilerplate, works perfectly with Expo
-

GLOBAL FOUNDATION (BUILD THIS FIRST)

1. Theme System

Before touching any screen, set up your theme system so dark/light mode works everywhere automatically.

```
/src/theme/  
  colors.ts      - all color tokens for  
dark + light  
  spacing.ts     - padding/margin scale  
  typography.ts  - font sizes, weights,  
families  
  index.ts       - exports useTheme() hook
```

colors.ts structure:

```
export const darkTheme = {  
  bg: {  
    primary: '#06050d',  
    secondary: '#0e0b1c',  
    tertiary: '#140f29',  
    card: '#0e0b1c',  
  },  
  text: {  
    primary: '#f4f2fb',  
    secondary: '#8d87ad',  
    accent: '#7c5cff',  
  },  
  accent: {  
    purple: '#7c5cff',  
    teal: '#3fe1c5',  
    orange: '#ff9a4d',  
  },  
  border: 'rgba(140,120,255,0.14)',  
  sent: '#7c5cff',          // sent bubble  
background  
  received: '#1a1730',      // received  
bubble background
```

```

}

export const lightTheme = {
  bg: {
    primary: '#f0f0f7',
    secondary: 'ffffff',
    tertiary: '#e8e6f0',
    card: 'ffffff',
  },
  text: {
    primary: '#0e0b1c',
    secondary: '#6b6880',
    accent: '#7c5cff',
  },
  accent: {
    purple: '#7c5cff',
    teal: '#2db89e',
    orange: '#e07a2a',
  },
  border: 'rgba(100,80,200,0.15)',
  sent: '#7c5cff',
  received: '#e8e6f4',
}

```

useTheme hook:

```

// reads from MMKV, listens for changes
const useTheme = () => {
  const isDark = useThemeStore(s =>
s.isDark)
  return isDark ? darkTheme : lightTheme
}

```

2. Zustand Store Setup

Create these stores before building any screen:

```
/src/store/
  useAuthStore.ts      – user session,
  profile
  useThemeStore.ts     – dark/light mode
  toggle
  useBubbleStore.ts    – selected chat
  bubble style
  useChatStore.ts      – active
  messages, reactions, typing
  useRoomStore.ts      – active rooms,
  room detail state
  useSocketStore.ts    – socket
  connection instance
  useSettingsStore.ts  – notifications,
  haptics, sound
```

useSocketStore.ts – CRITICAL:

```
// Single socket instance shared across the
// entire app
// Never create multiple socket connections
const useSocketStore = create((set, get) =>
({
  socket: null,
  connected: false,
  connect: (token) => {
    if (get().socket) return; // prevent
    duplicate connections
```

```

    const socket = io(BACKEND_URL, {
      auth: { token },
      transports: ['websocket'], // skip
polling for performance
      reconnection: true,
      reconnectionDelay: 1000,
    });
    socket.on('connect', () => set({
connected: true }));
    socket.on('disconnect', () => set({
connected: false }));
    set({ socket });
  },
  disconnect: () => {
    get().socket?.disconnect();
    set({ socket: null, connected: false
});
  }
}))

```

useBubbleStore.ts:

```

const BUBBLE_STYLES = [
  'minimal', 'glass', 'neon', 'gradient',
'liquid',
  'cyber', 'outline', 'shadow', 'pixel',
'hologram',
  'flame', 'cosmic'
]

const useBubbleStore = create(
  persist(
    (set) => ({

```

```
        selectedStyle: 'minimal',
        setStyle: (style) => set({
selectedStyle: style })),
    }),
    { name: 'bubble-style', storage:
mmkvStorage }
  )
)
```

3. Socket.io Event Map

Define ALL socket events before building screens so you know exactly what exists:

```
// EMIT (client → server)
'message:send'          // send a message
'message:react'         // add/remove
reaction
'message:reply'         // reply to a
message
'message:seen'          // mark messages as
seen
'typing:start'           // user started
typing
'typing:stop'           // user stopped
typing
'room:join'             // join a room's
socket room
'room:leave'            // leave a room's
socket room
'task:complete'         // complete a task
```



```
'thread:send'          // send message in
task thread

// LISTEN (server → client)
'message:new'          // incoming message
'message:reaction'     // reaction
added/removed
'message:status'       // status update
(delivered, seen)
'typing:update'        // someone
is/stopped typing
'room:activity'        // room-level
activity (task completed, member joined)
'thread:message'       // new message in
task thread
'user:online'          // friend came
online
'user:offline'         // friend went
offline
```

SCREEN BUILD ORDER

Reference images for every screen are in `/assets`. Look at the matching image before building each section — the plan describes the logic, the image shows the exact visual target.

Build in this exact order — each screen depends on what comes before it:

1. Theme system + stores (foundation)

2. Chat bubble component system
 3. Message status system
 4. Messages (hub) screen
 5. Individual chat screen
 6. Chat profile swipe-down panel
 7. Room detail screen
 8. Room chat (expanded standalone)
 9. Room onboarding flow
 10. Krios AI chat screen
 11. Settings screen
-

SCREEN 1 — MESSAGES HUB

Reference images: `assets/07-messages-hub.png` (main target), `assets/05-messages-flow-redesign.png` (structural rationale for why this layout was chosen over entering a room first)

 Messages Hub

 Messages Flow Redesign

Logic:

- On mount: fetch People (direct message contacts), Threads (task threads user is in), Room Activity (recent room updates) from Supabase

- Subscribe to socket events: `message:new`, `user:online`, `user:offline`, `room:activity`
- Unread count: stored in Zustand, updated by socket events
- Pinned conversations: stored in Supabase, fetched separately
- Search: filters People + Threads + Room Activity locally first, then queries Supabase if no local match
- The unread filter (badge showing "Unread 2") filters the Recent list to show only conversations with unread messages

Data structure:

```
type Conversation = {
  id: string
  type: 'direct' | 'group'
  name: string
  avatar?: string
  lastMessage: string
  lastMessageTime: string
  unreadCount: number
  isOnline: boolean
  isPinned: boolean
  isMuted: boolean
  isVerified?: boolean
  lastSender?: string // "You:" or
  "Olivia:" prefix
}
```

```
type Thread = {  
  id: string  
  taskTitle: string  
  roomName: string  
  lastMessage: string  
  lastMessageTime: string  
  unreadCount: number  
  taskIcon: string  
}  
  
type RoomActivity = {  
  id: string  
  roomName: string  
  roomIcon: string  
  activityText: string  
  time: string  
  unreadCount: number  
}
```

Performance notes:

- Use FlashList not FlatList for all lists
- Memoize each conversation row with React.memo
- Debounce search input by 300ms
- Load sections independently so Pinned shows immediately while People/Threads fetch

UI notes:

- Header: K logo left, "Messages" title with purple dot, filter icon + compose icon right
 - Search bar: full width, has advanced filter button (sliders icon) on the right
 - "Start a conversation" CTA card: glowing plus icon, tappable, goes to friend search
 - PINNED section: shows up to 3 pinned, "View all" if more
 - RECENT section: "Unread" badge filter top right of section
 - Muted conversations: show bell-off icon instead of unread count
 - Verified conversations: blue checkmark next to name (Krios Team)
 - Swipe left on conversation row: mute, pin, delete actions
-

SCREEN 2 — INDIVIDUAL CHAT

Reference images: `assets/08-chat-screen-reactions.png` (reactions, reply, plus tray), `assets/01-profile-panel.png` (swipe-down profile panel), `assets/02-chat-bubbles-basic.jpg` (baseline bubble reference), `assets/06-chat-bubble-styles.png` (all 12 bubble style options + status system)

 Chat Reactions

 Profile Panel

 Chat Bubbles Basic

 Bubble Styles

Logic (most complex screen):

Message loading:

- Load last 30 messages from Supabase on mount
- Load older messages when user scrolls to top (pagination)
- New messages arrive via socket `message:new`
- NEVER re-fetch all messages — only append new ones

Typing indicator:

- Emit `typing:start` when user starts typing, debounced
- Emit `typing:stop` after 2 seconds of no input
- Listen for `typing:update` to show/hide "..." bubble

Message status flow:

```
SENDING → (socket confirms) → SENT →  
(recipient receives) → DELIVERED →  
(recipient opens chat) → SEEN
```

Each status is a socket event. Status only goes forward, never backward.

Reactions:

- Long press on any bubble → reaction picker appears (emoji row overlay)
- Tap emoji → emits `message:react` with `messageId` + emoji
- If user already reacted with same emoji → removes reaction
- Multiple users can react with same emoji → count shows below bubble
- Reactions stored in Supabase messages table as JSON array

Reply system:

- Swipe right on any bubble → triggers reply mode
- Reply bar appears above input showing preview of replied message
- Sent message contains `replyTo: { id, text, senderName }` field
- Rendered as a quoted box above the bubble text

Plus tray:

- Plus button pressed → tray slides up from below input
- Items: Gallery, File, Voice, Location, Contact

- Gallery → expo-image-picker
- File → expo-document-picker
- Voice → expo-av recording
- Location → expo-location, sends as special message type
- Contact → contacts picker

Profile swipe-down panel:

- The chat screen has TWO layers stacked:
 - Layer 1 (bottom): profile card content (banner image, avatar, name, stats, actions)
 - Layer 2 (top): the actual chat (header + messages + input)
- Layer 2 (chat) is a PanGestureHandler with translateY animated value
- When user drags DOWN on the chat layer, it slides down revealing the profile layer underneath
- A drag handle pill sits at the top of the chat layer as the visual cue
- The chat header stays fixed — only the chat body slides
- When released past 40% of screen height → snaps to open (profile visible)
- When released below 40% → snaps back closed


```
// Reanimated logic for swipe-down panel
const translateY = useSharedValue(0)
const MAX_TRANSLATE = SCREEN_HEIGHT * 0.65

const gesture = Gesture.Pan()
  .onUpdate((e) => {
    translateY.value = Math.max(0,
Math.min(e.translationY, MAX_TRANSLATE))
  })
  .onEnd((e) => {
    if (e.translationY > SCREEN_HEIGHT *
0.3) {
      translateY.value =
withSpring(MAX_TRANSLATE)
    } else {
      translateY.value = withSpring(0)
    }
  })
})
```

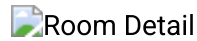
UI notes for individual chat:

- Header: back arrow, avatar + name + online status, call icon, video icon, three dots
- Date dividers: "Today", "Yesterday", "Monday 15 Jun" — center aligned, pill shaped
- Sent bubbles: right aligned, purple colored, time + status ticks inside
- Received bubbles: left aligned, dark colored, sender name above (for group contexts)

- Typing indicator: three animated dots in received bubble style
 - Reaction display: small emoji + count overlaid below bubble
 - Reply preview inside bubble: quoted box with left border, truncated text, sender name
-

SCREEN 3 — ROOM DETAIL

Reference image: `assets/03-room-detail.png`



Logic:

- On mount: fetch room data, members, tasks for selected day, recent chat messages
- Calendar strip: fetch task completion data for current week, show dot indicators
- Tapping a day on calendar: filters tasks to that date
- Activity pill: listens to `room:activity` socket events, shows last event
- Task cards: fetch from Supabase filtered by room + date, sorted Active then Pending
- Collapsible chat section:

- Collapsed state: shows last 2 messages as preview rows
- Expanded state: full chat list + input (uses the room chat screen inline)
- Collapse/expand triggered by chevron tap OR scrolling up past the section
- Member completion rings: fetched from Supabase – which members completed all tasks today

Room onboarding trigger logic:

```
// On entering a room, check if user has
seen onboarding for this room
const hasSeenOnboarding = await
mmkv.getString(`onboarded_${roomId}`)
if (!hasSeenOnboarding) {
  setShowOnboarding(true)
}
// After completing:
mmkv.set(`onboarded_${roomId}`, 'true')
```

Data to fetch on room entry:

```
// Parallel fetches, not sequential
const [room, members, tasks, chatPreview] =
await Promise.all([
  fetchRoom(roomId),
  fetchRoomMembers(roomId),
```

```
fetchRoomTasks(roomId, selectedDate),  
fetchRoomChatPreview(roomId, limit: 2),  
])
```

UI notes:

- Background image header: room's cover image with dark overlay gradient
 - Room icon: rounded square with icon/emoji, overlaid on header
 - Room name: large bold text, member count subtitle
 - Member avatars row: stacked circles, +5 overflow indicator, "Room settings" button
 - Calendar strip: same style as home screen calendar
 - Activity pill: green dot, username + action text, "View activity" link right
 - Task cards: left colored border, icon, task name, tags (ACTIVE/PENDING), member avatars, progress bar, percentage
 - Chat section header: "Chat 12" with "See all" right — tapping "See all" goes to expanded room chat
 - Chat preview rows: avatar, name, last message, timestamp
-

SCREEN 4 – ROOM CHAT (EXPANDED)

Reference image: `assets/09-room-chat-expanded.png`



Logic:

- Same message logic as individual chat but scoped to room + task thread
- Header shows room name (not person name) + LIVE green indicator
- All members of the room can see messages
- Sender name shown above each received bubble (since multiple people)
- Waveform/audio icon in header top right (for future voice note feature – include button, disable for now)
- Messages fetched from Supabase filtered by roomId + threadId
- Socket room:

```
room:${roomId}:thread:${threadId}
```

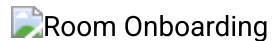
UI differences from individual chat:

- Pure black background (#000000 or very near it)

- Sent bubbles: outline style (dark fill + purple glow border) – this is the default for room chat regardless of user's bubble style setting
 - Received bubbles: dark card style with sender name label above in purple
 - Header: back arrow, room avatar circle with initial, room name + LIVE badge, waveform icon, three dots
 - Input: plus button (left), text input, emoji icon, send button (right)
-

SCREEN 5 – ROOM ONBOARDING FLOW (5-step modal)

Reference image: `assets/04-room-onboarding.png`



Logic:

- Renders as a full-screen modal over room detail
- Controlled by step index (0–4)
- Step data fetched before modal opens (room stats, members, etc.)
- Interests selection: stored locally in Zustand then saved to Supabase on step 4

- Recommended tasks: generated from selected interests
- "Join X Challenges" button: bulk-joins selected tasks
- "Skip" on step 4: skips task joining, goes straight to step 5
- Completing step 5: saves onboarding flag to MMKV, closes modal with animation

Step data:

Step 1 – Welcome:

- Room name, banner image
- Stats: active members count, active challenges count, completion rate

Step 2 – Members:

- Top 3-4 members with recent achievements
- "+X others" overflow count
- Each member shows their standout stat (streak, tasks completed, etc.)

Step 3 – Interests:

- List of interest tags with icons
- Multi-select checkboxes
- "You can update this anytime in room settings" footnote

Step 4 – Recommended Tasks:

- Generated from interests selected in

step 3

- Checkboxes to select which to join
- "Join X Challenges" button (count updates as user selects)
- "Skip" text button

Step 5 – You're In:

- Confirmation illustration / animation
- "Live in the room" feed showing real-time member activity
- "Enter Room" button
- "Maybe later" text button

Animation between steps:

- Slide left for forward, slide right for back
- Progress bar at top animates width based on step index
- Each step fades in

SCREEN 6 – KRIOS AI CHAT

Reference image: `assets/10-krios-ai-chat.png`

(left = active state, right = empty state)

 Krios AI Chat

Logic:

- Separate from regular chat — has its own screen and route
- Messages stored locally in Zustand (not in Supabase by default — privacy)
- API calls go to Vercel AI SDK endpoint
- Message flow: user types → emit to Vercel endpoint → stream response back
- Streaming: show AI response word by word as it arrives (streaming API)
- Waveform icon in header: toggles voice mode (for future)
- Context aware: Krios has access to user's task list and room data to give relevant responses
- Starter prompts: shown on empty state, tapping fills input and sends

Empty state:

- Planet mascot (import your animated image here)
- "Start a conversation" heading
- Sub text
- 3 starter prompt chips

Active state:

- Krios messages: sparkle avatar, dark bubble with waveform indicator showing it was voice-ready

- User messages: standard sent bubble in their selected bubble style
- Typing indicator while AI is generating

System prompt to send to Vercel AI:

Include in every request:

```
const systemPrompt = `
You are Krios, an AI built into a habit
tracking app.
User's name: ${user.name}
Today's tasks:
${JSON.stringify(todaysTasks)}
Active rooms:
${JSON.stringify(activeRooms)}
Respond concisely. Max 3 sentences unless
the user is struggling.
Never moralize. Offer one next step at a
time.
`
```

SCREEN 7 – SETTINGS

Reference images: `assets/11-settings-option1.png` (recommended – mobile scroll layout), `assets/12-settings-option2.png` (sidebar concept, reference only – not the primary build target on phone)

 Settings Option 1

 Settings Option 2

Logic:

- All settings persist via MMKV (fast, synchronous)
- Theme toggle: calls `useThemeStore.toggle()`, immediately applies across app
- Bubble style: opens a sub-screen showing all 12 bubble previews as a scrollable carousel with live preview
- Navigation style: toggles between bottom tab bar and sidebar panel — requires app restart or dynamic nav re-render
- Smart Reminders: schedules local notifications via expo-notifications
- Sound Effects: flag stored in MMKV, checked before playing any sound
- Haptic Feedback: flag stored in MMKV, checked before any haptics call
- Edit Profile: navigates to profile edit screen

Bubble style picker sub-screen:

- Grid of all 12 bubble styles
- Each shows a preview of "hi 9:04 AM ✓✓" rendered in that style
- Currently selected has purple border highlight

- Tapping immediately updates useBubbleStore
- Change is instant – no save button needed

Settings structure:

NAVIGATION

Bottom Tab Bar (radio select)

Sidebar Panel (radio select)

APPEARANCE

Dark Mode (toggle)

Chat Bubble Style (navigates to picker, shows current style name + mini preview)

NOTIFICATIONS

Smart Reminders (toggle) – morning digest 8am, evening preview 8pm, due task alerts

Sound Effects (toggle)

Haptic Feedback (toggle)

ACCOUNT

Edit Profile (chevron)

Privacy & Security (chevron)

Blocked Users (chevron)

DANGER

Clear Chat History (red, confirmation required)

Log Out (red, confirmation required)

Delete Account (red, double confirmation required)

CHAT BUBBLE SYSTEM

Reference images: `assets/06-chat-bubble-styles.png` (all 12 styles), `assets/13-flame-bubble-asset.png` (the actual flame PNG asset – use directly as `ImageBackground`)

 Flame Bubble Asset

Create a single `ChatBubble` component that renders different styles based on the `bubbleStyle` prop.

```
/src/components/chat/  
  ChatBubble.tsx      – master  
component, switches style  
  bubbles/  
    MinimalBubble.tsx  
    GlassBubble.tsx  
    NeonBubble.tsx  
    GradientBubble.tsx  
    LiquidBubble.tsx  
    CyberBubble.tsx  
    OutlineBubble.tsx  
    ShadowBubble.tsx  
    PixelBubble.tsx  
    HologramBubble.tsx  
    FlameBubble.tsx  
    CosmicBubble.tsx
```

ChatBubble.tsx master switch:

```

const ChatBubble = ({ message, isSent,
bubbleStyle }) => {
  const BubbleComponent = {
    minimal: MinimalBubble,
    glass: GlassBubble,
    neon: NeonBubble,
    gradient: GradientBubble,
    liquid: LiquidBubble,
    cyber: CyberBubble,
    outline: OutlineBubble,
    shadow: ShadowBubble,
    pixel: PixelBubble,
    hologram: HologramBubble,
    flame: FlameBubble,
    cosmic: CosmicBubble,
  }[bubbleStyle] || MinimalBubble

  return (
    <BubbleComponent
      text={message.text}
      time={message.createdAt}
      status={message.status}
      isSent={isSent}
      replyTo={message.replyTo}
      reactions={message.reactions}
    />
  )
}

```

Every bubble component receives the same props:

```
type BubbleProps = {
  text: string
  time: string
  status: 'sending' | 'sent' | 'delivered'
  | 'seen'
  isSent: boolean
  replyTo?: { text: string; senderName:
string } | null
  reactions?: { emoji: string; count:
number; userReacted: boolean }[]
}
```

Key bubble notes:

- Flame bubble uses the PNG image you have as ImageBackground
- Glass bubble: backdrop-filter blur effect using react-native-blur
- Neon bubble: purple glow via shadow (elevation on Android, shadow props on iOS)
- Liquid bubble: organic borderRadius animation using Reanimated
- Cyber bubble: sharp corners with angular cut effect using SVG or border tricks
- Pixel bubble: chunky border, monospace font
- Hologram bubble: teal/blue gradient + shimmer animation
- Cosmic bubble: dark gradient + star particles using Canvas or absolute dots

MESSAGE STATUS SYSTEM

Status icons:

- Sending – hollow circle (clock)
- ✓ Sent – single grey tick
- ✓✓ Delivered – double grey tick
- ✓✓ Seen – double purple/blue tick (color change)
- 👁 Played – eye icon (for voice notes)
- 😊 Reacted – smiley icon
- ↩ Replied – reply arrow icon
- 📌 Pinned – pin icon

Status component:

```
const MessageStatus = ({ status, isRead })
=> {
  if (status === 'sending') return
  <ActivityIndicator size="small" />
  if (status === 'sent') return <Icon
name="check" color="grey" size={12} />
  if (status === 'delivered') return
  <DoubleCheck color="grey" size={12} />
  if (status === 'seen') return
  <DoubleCheck color="#7c5cff" size={12} />
  return null
}
```

Socket flow for status:


```
1. User sends message → optimistically set
status: 'sending'
2. Server confirms receipt → emit
'message:status' { id, status: 'sent' }
3. Recipient's device connects/receives →
server emits to sender: { id, status:
'delivered' }
4. Recipient opens the conversation →
client emits 'message:seen' → server
broadcasts to sender: { id, status: 'seen'
}
```

PERFORMANCE RULES (LOW-BUDGET ANDROID)

These are non-negotiable for your target devices:

1. FlashList everywhere

Replace every FlatList with FlashList from
[@shopify/flash-list](https://github.com/Shopify/flash-list).

```
import { FlashList } from "@shopify/flash-
list"
// estimatedItemSize is required – measure
your average item height
<FlashList data={messages}
estimatedItemSize={72} renderItem={...} />
```

2. Memoize all list items

```
const MessageRow = React.memo(({ message })  
=> { ... })
```

3. Keep all tabs mounted (pager-view)

For the main tab navigation, use react-native-pager-view with `offscreenPageLimit` :

```
<PagerView offscreenPageLimit={1}  
initialPage={0}>  
  <HomeTab />  
  <MessagesTab />  
  <RoomsTab />  
  <MoreTab />  
</PagerView>
```

This keeps adjacent tabs mounted so swiping feels instant.

4. Single socket connection

Never create more than one socket connection.
Create it in useSocketStore on app launch, share it everywhere. Disconnect on logout only.

5. Image optimization

```
import { Image } from 'expo-image' // NOT  
React Native Image
```

```
// expo-image has built-in caching,  
blurhash placeholders, and faster loading  
<Image source={{ uri: url }} placeholder=  
{blurhash} contentFit="cover" />
```

6. Avoid re-renders

- All Zustand selectors should be specific:
`useStore(s => s.specificField)` not
`useStore()`
- Avoid putting large objects in state that changes frequently
- For message list, only append new messages — never reset the entire array

7. Haptics cost nothing

```
import * as Haptics from 'expo-haptics'  
// Only fire if setting is enabled  
if (hapticsEnabled)  
  Haptics.impactAsync(Haptics.ImpactFeedbackS  
tyle.Light)
```

8. Animations on UI thread

All Reanimated animations must use

`useAnimatedStyle` and `useSharedValue` — never
Animated from React Native core. Reanimated runs
on the UI thread and won't drop frames even on low-
end devices.

FOLDER STRUCTURE

```
/app
  (auth)/
    login.tsx
    register.tsx
  (tabs)/
    index.tsx          – Home
    messages.tsx       – Messages Hub
    rooms.tsx          – Rooms
    more.tsx           – More/Profile
  chat/[id].tsx        – Individual Chat
  room/[id].tsx        – Room Detail
  room/[id]/chat.tsx   – Room Chat Expanded
  ai-chat.tsx          – Krios AI Chat
  settings.tsx         – Settings
  settings/bubble-picker.tsx

/src
  /components
    /chat
      ChatBubble.tsx
      MessageRow.tsx
      MessageStatus.tsx
      ReactionPicker.tsx
      ReplyBar.tsx
      PlusTray.tsx
      TypingIndicator.tsx
    /bubbles           – all 12 bubble
  components
    /room
```

```
RoomCard.tsx
TaskCard.tsx
MemberAvatar.tsx
CalendarStrip.tsx
ActivityPill.tsx
CollapsibleChat.tsx
/onboarding
  OnboardingModal.tsx
  StepWelcome.tsx
  StepMembers.tsx
  StepInterests.tsx
  StepTasks.tsx
  StepYoureIn.tsx
/profile
  ProfilePanel.tsx  – the swipe-down
profile card
/ui
  Button.tsx
  Avatar.tsx
  Badge.tsx
  Toggle.tsx
  Divider.tsx
/store
  useAuthStore.ts
  useThemeStore.ts
  useBubbleStore.ts
  useChatStore.ts
  useRoomStore.ts
  useSocketStore.ts
  useSettingsStore.ts
/hooks
  useSocket.ts
  useKeyboard.ts
  useTheme.ts
```

```
useMessages.ts
useRoomTasks.ts
/services
  supabase.ts
  api.ts
  socket.ts
/theme
  colors.ts
  typography.ts
  spacing.ts
  index.ts
/utils
  time.ts
  format.ts
  haptics.ts
  storage.ts          – MMKV helpers
```

BUILD ORDER — WEEK BY WEEK

Phase 1: Foundation (do this first, touch no screens yet)

- ☐ Install all missing packages
- ☐ Set up theme system (colors, useTheme hook)
- ☐ Set up all Zustand stores
- ☐ Set up MMKV storage helpers
- ☐ Set up single Socket.io connection in useSocketStore
- ☐ Set up Supabase client

- ☐ Define all socket event types

Phase 2: Chat Bubble System

- ☐ Build MinimalBubble (this is your base)
- ☐ Build MessageStatus component
- ☐ Build ReactionPicker
- ☐ Build ReplyBar
- ☐ Build master ChatBubble switcher
- ☐ Build remaining 11 bubble styles one at a time
- ☐ Test all bubbles on low-budget Android

Phase 3: Core Screens

- ☐ Messages Hub screen
- ☐ Individual Chat screen (no profile panel yet)
- ☐ Wire up Socket.io for real-time messages
- ☐ Add typing indicator
- ☐ Add message status updates
- ☐ Add reactions
- ☐ Add reply system
- ☐ Add plus tray

Phase 4: Profile Panel + Room Screens

- ☐ Profile swipe-down panel (Reanimated gesture)
- ☐ Room Detail screen
- ☐ Room Chat expanded screen

- ☐ Collapsible chat section in Room Detail

Phase 5: Onboarding + AI

- ☐ Room Onboarding 5-step modal
- ☐ Krios AI Chat screen (empty + active states)
- ☐ Wire Vercel AI SDK

Phase 6: Settings + Polish

- ☐ Settings screen
 - ☐ Bubble style picker screen
 - ☐ Dark/light mode tested on all screens
 - ☐ Performance audit on low-budget Android
 - ☐ Final socket event testing
-

NOTES

- The "profile bottom sheet" you referred to is actually a swipe-DOWN reveal — the chat slides down to reveal the profile underneath.
@gorhom/bottom-sheet by default opens from the bottom but can be used inverted. Alternatively build it purely with Reanimated PanGestureHandler for full control.
- Room task thread chat (in-task chat) follows the same logic as Room Chat expanded but scoped to a specific task thread ID.

- Room chat uses Outline bubble style always, regardless of user's personal bubble preference.
- Krios AI chat uses its own bubble style (dark card with waveform indicator for AI, user's selected style for user messages).
- Android keyboard behavior differs from iOS — test `react-native-keyboard-controller` on both. Use `KeyboardAwareScrollView` as fallback if needed.